Introduction to

Programming with Python

Libraries

made by CS Made Easy

Do we really have to write everything from scratch?

Problem:

We want to simulate a coin flip.

Q:Do we need to build randomness ourselves?

What is a library?

- A library is code written by other people that we can reuse.
- A module is one part of a library or one file of reusable code.

Note: Instead of reinventing a solution, we borrow from others.

Let us solve the coin flip problem

Problem: How can we simulate heads or tails?

```
import random

coin = random.choice(["heads", "tails"])
print(coin)
```

Explanation:

- import random brings in the module
- choice() picks one item from a list

Why not write our own randomness?

- We could try, but that would be harder and less reliable.
- Python's random module already gives us tested tools.

Message:

- Use built-in tools when they already solve the problem well.

Another problem — shuffling cards

Problem: How do we shuffle a deck of cards?

```
import random

cards = ["A", "2", "3", "4", "5"]
random.shuffle(cards)
print(cards)
```

Explanation:

- shuffle() changes the list order
- This solves a real problem in a simple way

Why import random and not just choice?

- random is the module name.
- choice is a function inside that module.
- The module name helps Python know where the function came from.

Using only one part of a module

```
from random import choice
coin = choice(["heads", "tails"])
print(coin)
```

- This lets us use choice() directly.

When do we use from ... import ...?

Sometimes we only need one function.
Then from ... import ... can make code shorter and cleaner.

Problem

In a science experiment, temperature readings may vary. Let's simulate one temperature reading between 20°C and 25°C.

randint()

randint()

- The randint() function is used to generate a random integer number within a specific range.

```
import random
# Generates a number between a and b
random.randint(a, b)
```

Key Rules

- Inclusive: Unlike most Python functions, it includes both the start and the end numbers.
- Integers Only: It only returns integer numbers (no decimals).
- Requirement: The first number must be less than or equal to the second number.

Solution to Problem

```
import random

temperature = random.randint(20, 25)
print(temperature)
```

But ...

"A real experiment needs many readings, not just one."

```
import random

temps = []

for i in range(5):
    temps.append(random.randint(20, 25))
print(temps)
```

Output:

[21,23,21,24,25]

Analyzing with Statistics module

The statistics module provides functions for calculating mathematical statistics

Core Functions

- mean(): Calculates the arithmetic average of data.
- median(): Finds the middle value (robust against outliers).
- mode(): Returns the most frequent value in a dataset.
- stdev(): Calculates the standard deviation (measure of data spread).
- variance(): Measures how far a set of numbers are spread out from their average.

statistics.mean

```
import statistics
temps = [21,23,21,24,25]
print(statistics.mean(temps))
```

Output:

```
22.8
```

So far, our programs work...

But how else could we get input without writing it inside the code?

Q:We can pass input when we run the program?

Yes, We can pass input when we run the program

```
>_
  python program.py Abel
```

Note:

Now the program receives input from outside

sys

- The sys module gives us access to information about how the program was run.

```
import sys
print(sys.argv)
```

Q:What is sys.argv?

key idea:

The command line gives our program input before it even starts running.

Understanding sys.argv

- sys.argv is a list.
- sys.argv[0] → the program name
- sys.argv[1] → the first input from the use

key idea:

- The command line gives our program input before it even starts running.

Saying hello to a user

Version-1

```
import sys

if len(sys.argv) < 2:
    print("Too few arguments")
elif len(sys.argv) > 2:
    print("Too many arguments")
else:
    print("Hello,", sys.argv[1])
```

Saying hello to a user

Version-2

```
import sys

if len(sys.argv) < 2:
    print("Too few arguments")
elif len(sys.argv) > 2:
    print("Too many arguments")

print("Hello,", sys.argv[1])
```

Q: What do u think sys.exit is doing?

What is sys.exit()?

- sys.exit() ends the program right away
- You can provide a message that explains why it stopped
- Any code after sys.exit() will not run

Beyond "Boring" Greetings

The Problem:

- We want our program to do something "cool"—like showing a cow or a dragon delivering a message—but we don't want to manually draw ASCII art character-by-character.

Q: Should we just print an actual image file?

Third-Party Library

- cowsay is a that allows reuse someone else's code to generate complex visuals instantly.

```
import cowsay
# saying hello to a user
cowsay.cow("hi user")
```

Q: but is cowsay a module or a libary ?

Modules vs. Packages

- Module: A single .py file with functions/classes (e.g., math.py).
- Package: A collection of modules organized in a folder with __init__.py.
- Example: cowsay has multiple modules for different animals.

Third-party packages:

- Developers share reusable code so you don't have to reinvent the wheel.

Installing Packages with pip:

- Python is powerful, but it does not come with third-party libraries pre-installed.
- To get them, we use a tool called pip.
- What is pip? It is the standard package manager for Python. It downloads and installs software from the Python Package Index (PyPI).

Command Line Instructions

(Terminal/CMD):

- Verify Installation: Check if you have pip ready to go.

```
>_ pip --version
```

- Download a Package: Get the cowsay library from the internet.

```
>_ pip install cowsay
```

- Stay Updated: If a library adds new features (like more animals!), upgrade it.

```
>_ pip install --upgrade cowsay
```

Greeting using cowsay

```python
import sys
import cowsay
# Check if user provided a name
if len(sys.argv) != 2:
    sys.exit("Usage: python greet.py [your_name]")
# Get the user's name from command-line arguments
user_name = sys.argv[1]

# Make the cow greet the user
cowsay.cow(f"Hello, {user_name}! Welcome to Python!")
```

how to run: `python greet.py Tsion`

API

Why Do We Need APIs?

What if our program needs real music data?

What if we want:

- songs by an artist
- album names
- release dates

We cannot store all music data ourselves.

aha .. This is where APIs come in.

What Is an API?

- API = Programs talking to each other
- An API lets our Python program ask another system for data.

Example idea:

- Ask iTunes:
  "Give me songs by a specific artist"

Simple definition:

- "A messenger that takes your request, asks the appropriate server for the right information, and brings the data back to you."

but why not store the music ourselves?

- Millions of songs exist
- Data changes constantly
- Too much to manage manually

We cannot write every song !

Request and Response

How APIs work ?

- The Request (The "Ask"): Your program acts as the Client and sends a specific URL to the API.
  - Example: https://itunes.apple.com/search?term=jack+johnson
- The Processing (The "Work"): The API receives your request and finds the matching songs in Apple's database.
  - Action: The API filters millions of files to retrieve only the artist you requested.
- The Response (The "Answer"): The API sends back a response containing the data or a status message.
  - Example: A list of song titles, prices, and album art sent back to your app.

What Does the Response Look Like?

APIs return JSON

The iTunes API sends data in JSON format.

Example:

```
{
  "artistName": "Drake",
  "trackName": "God's Plan"
}
```

Get songs by any artist

idea:

- Let the user type an artist name

→ Get real songs from iTunes

But how do we do that ?

Note:

Python cannot "talk" to the internet on its own.

“surely Someone should have solved this problem sometime ago.”

The requests Module

- The requests module acts like a Web Browser inside your code.
- It packages your artist's name into a URL and sends it to Apple’s server.
- Apple’s Web API acts as the gatekeeper, taking your request and sending back the song data.

The API Workflow

- Request: requests.get("https://itunes.apple.com/...")
- Process: The Web API asks Apple's server for the specific artist.
- Response: The API delivers a list of songs back to your Python program.

The Digital Messenger

Component	Role	What it Does
Python requests	The **Web Client**	Sends the "Order" to the internet.
iTunes Web API	The **Messenger**	Asks the server and brings data back.
Apple Server	The Database	Stores millions of songs and prices.

Using requests with iTunes API

```python
import requests
import sys
artist = sys.argv[1]

url = f"https://itunes.apple.com/search?term={artist}&limit=5"
response = requests.get(url)

print(response.json())
```

Understanding:

- requests.get(url) → sends request
- url → contains the artist name
- response → stores the API reply
- .json() → converts to Python data

"We are now working with real data from the internet."

Q:how do Extract Useful Data like getting song names?

```python
import requests
import sys
artist = sys.argv[1]

url = f"https://itunes.apple.com/search?term={artist}&limit=5"
response = requests.get(url)

data = response.json()

for result in data["results"]:
  print(result["trackName"])
```

Example result:

Run:

```
>_ python program.py Drake
```

Output:

God's Plan
One Dance
Hotline Bling
Started From The Bottom
In My Feelings

take away:

APIs connect our programs to the real world.

Open_AI API

Introduction to

Programming with Python

Libraries